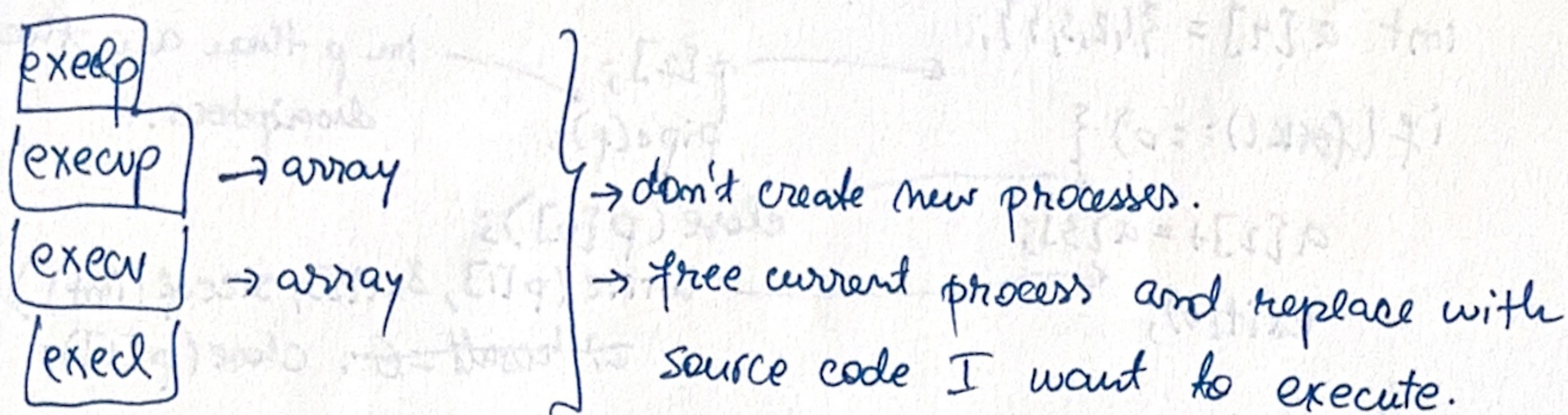




argv[0] → name of program

execvp

"grep",
"grep", ←
"-E",

"/aml/..."

execl

"/bin/grep",
"/bin/grep", ←
"-E",

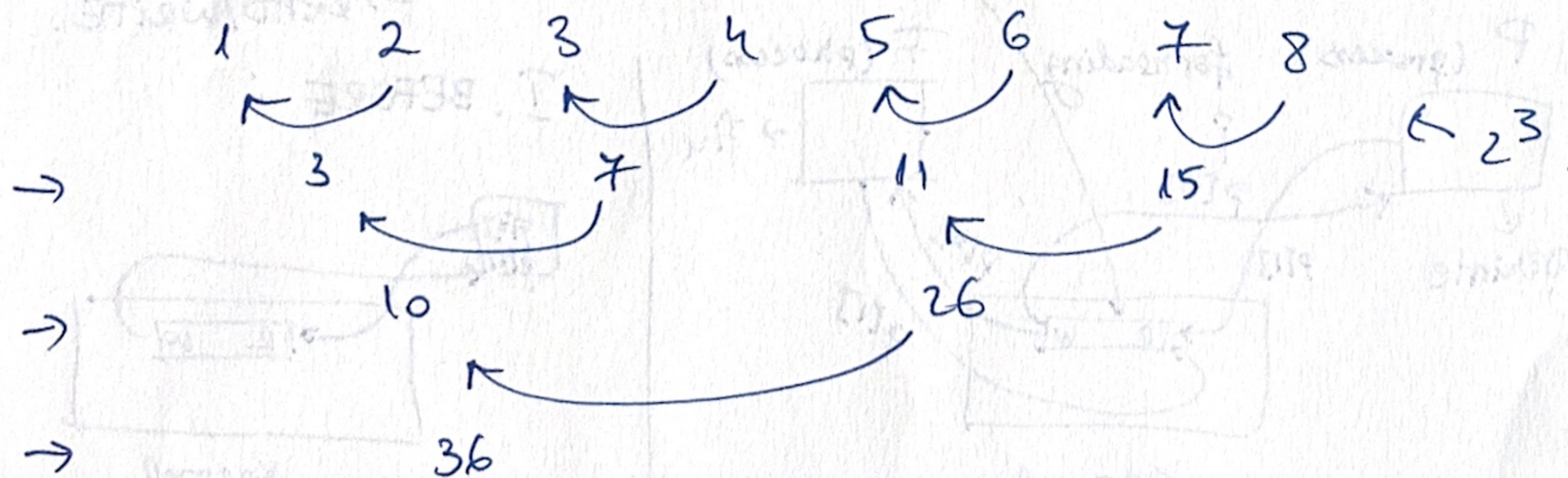
!

PROGRAM 1: PROCESS BRANCHING

fork() → duplicate child process

exec() → replace running code with "/bin/echo"

Problem: Sum of elements of an array of 1000000 elems.



3 steps:


```
int main (...) {
```

```
    int a[4] = {1, 2, 3, 4};
```

```
    if (fork() == 0) {
```

```
        a[2] += a[3];
```

```
        exit(0);
```

```
    }
```

```
    a[0] += a[1];
```

```
    wait(0);
```

```
    a[0] += a[2];
```

```
    printf (... a[0]);
```

```
    (void)...
```

```
    p[2];
```

```
    pipe(p);
```

```
    close(p[0]);
```

```
    write(p[1], &a[2], sizeof(int));
```

```
    // result = 6, close(p[1]);
```

```
    close(p[1]);
```

```
    read(p[0], &a[2], sizeof(int));
```

```
    close(p[0]);
```

=> result = 6.

close two times,
but in $\neq$ processes
→ close only the
border accesses to
the pipe.

!!
to be added so
the program works.

mechanisms for IPC

- pipe

- FIFO

- SHM (shared memory)

- MSG (transfers data) → message queues.

- SEM (semaphores)

→ we created a PIPE (unidirectional)

→ needs to be accessed

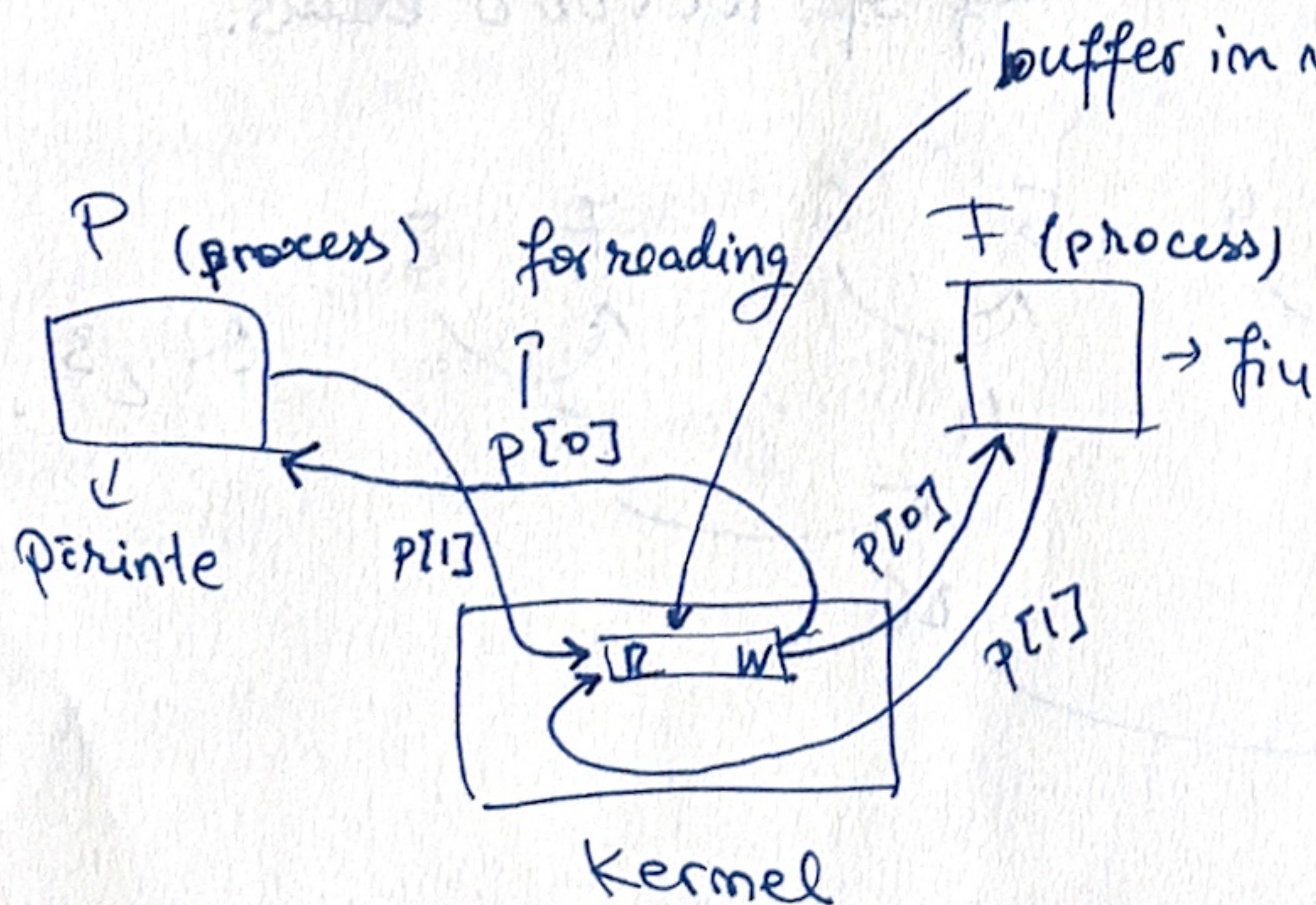
→ PIPE has no name, needs to
be inherited from somewhere
(mostenit)

→ feara .

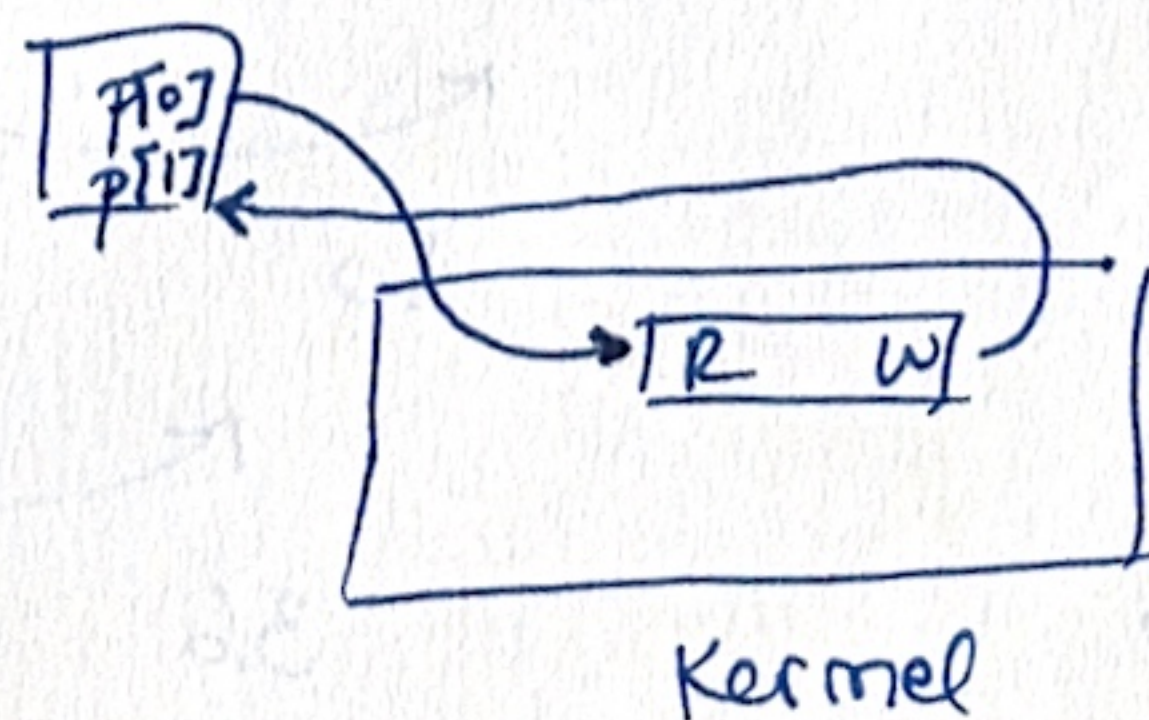
→ buffer overflow?

→ PIPE ~~live~~ lives in
memory.

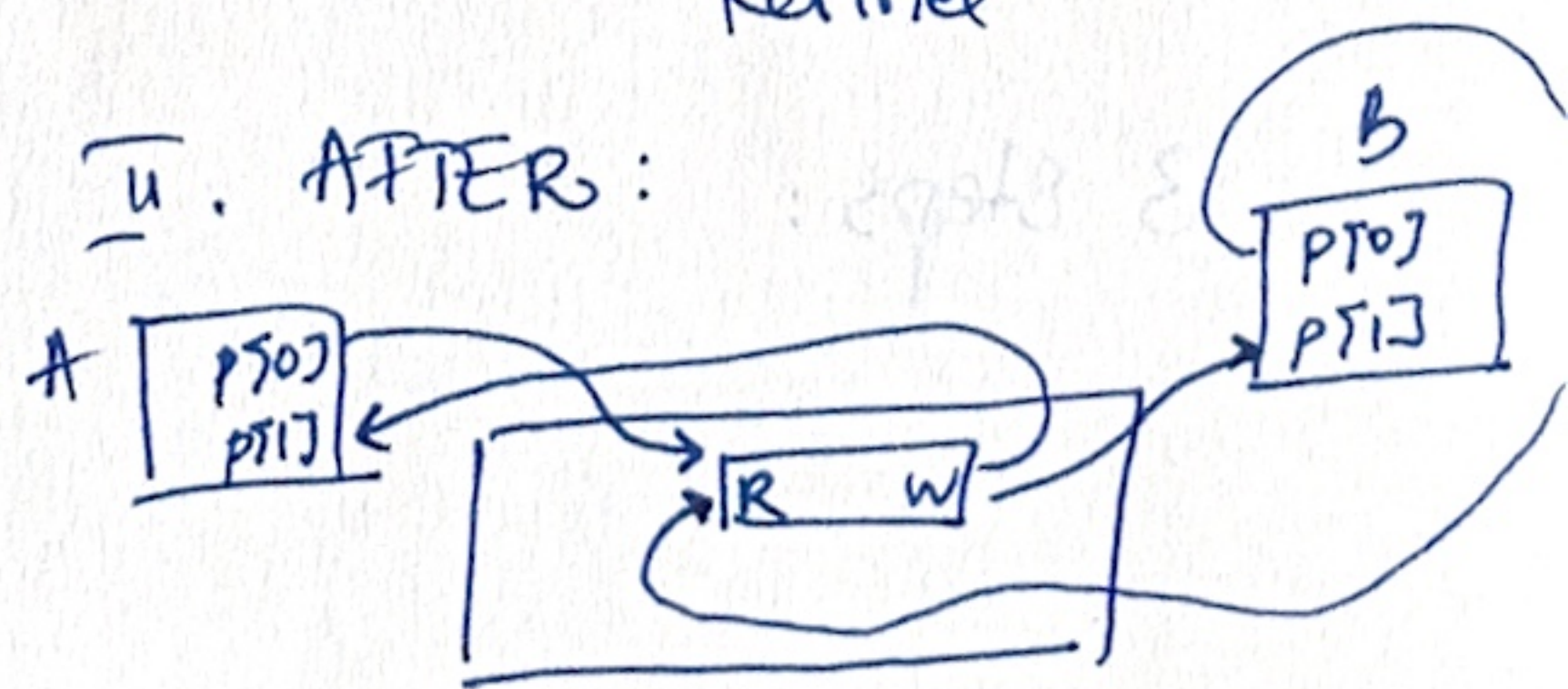
→ READ/WRITE.



I. BEFORE:



II. AFTER:



p[0] → reads

p[1] → writes

• Read from empty pipe.